

9

1.2 节练习

练习 1.3: 编写程序, 在标准输出上打印 Hello, World。

练习 1.4: 我们的程序使用加法运算符+来将两个数相加。编写程序使用乘法运算符*, 来打印两个数的积。

练习 1.5: 我们将所有输出操作放在一条很长的语句中。重写程序, 将每个运算对象的打印操作放在一条独立的语句中。

练习 1.6: 解释下面程序片段是否合法。

```
std::cout << "The sum of " << v1;
          << " and " << v2;
          << " is " << v1 + v2 << std::endl;
```

如果程序是合法的, 它输出什么? 如果程序不合法, 原因何在? 应该如何修正?

1.3 注释简介

在程序变得更复杂之前, 我们应该了解一下 C++ 是如何处理注释 (comments) 的。注释可以帮助人类读者理解程序。注释通常用于概述算法, 确定变量的用途, 或者解释晦涩难懂的代码段。编译器会忽略注释, 因此注释对程序的行为或性能不会有任何影响。

虽然编译器会忽略注释, 但读者并不会。即使系统文档的其他部分已经过时, 程序员也倾向于相信注释的内容是正确可信的。因此, 错误的注释比完全没有注释更糟糕, 因为它会误导读者。因此, 当你修改代码时, 不要忘记同时更新注释!

C++ 中注释的种类

C++ 中有两种注释: 单行注释和界定符对注释。单行注释以双斜线 (//) 开始, 以换行符结束。当前行双斜线右侧的所有内容都会被编译器忽略, 这种注释可以包含任何文本, 包括额外的双斜线。

另一种注释使用继承自 C 语言的两个界定符 (/* 和 */)。这种注释以 /* 开始, 以 */ 结束, 可以包含除 */ 外的任意内容, 包括换行符。编译器将落在 /* 和 */ 之间的所有内容都当作注释。

注释界定符可以放置于任何允许放置制表符、空格符或换行符的地方。注释界定符可以跨越程序中的多行, 但这并不是必须的。当注释界定符跨越多行时, 最好能显式指出其内部的程序行都属于多行注释的一部分。我们所采用的风格是, 注释内的每行都以一个星号开头, 从而指出整个范围都是多行注释的一部分。

10

程序中通常同时包含两种形式的注释。注释界定符对通常用于多行解释, 而双斜线注释常用于半行和单行附注。

```
#include <iostream>
/*
 * 简单主函数:
 * 读取两个数, 求它们的和
 */
int main()
{
```

```
// 提示用户输入两个数
std::cout << "Enter two numbers:" << std::endl;
int v1 = 0, v2 = 0;      // 保存我们读入的输入数据的变量
std::cin >> v1 >> v2;    // 读取输入数据
std::cout << "The sum of " << v1 << " and " << v2
               << " is " << v1 + v2 << std::endl;
return 0;
}
```



在本书中，我们用楷体来突出显示注释。在实际程序中，注释文本的显示形式是否区别于程序代码文本的显示，依赖于你所使用的程序设计环境是否提供这一特性。

注释界定符不能嵌套

界定符对形式的注释是以/*开始，以*/结束的。因此，一个注释不能嵌套在另一个注释之内。编译器对这类问题所给出的错误信息可能是难以理解、令人迷惑的。例如，在你的系统中编译下面的程序，就会产生错误：

```
/*
 * 注释对/* */不能嵌套。
 * “不能嵌套”几个字会被认为是源码，
 * 像剩余程序一样处理
 */
int main()
{
    return 0;
}
```

我们通常需要在调试期间注释掉一些代码。由于这些代码可能包含界定符对形式的注释，因此可能导致注释嵌套错误，因此最好的方式是用单行注释方式注释掉代码段的每一行。

```
// /*
// * 单行注释中的任何内容都会被忽略
// * 包括嵌套的注释对也一样会被忽略
// */
```

1.3 节练习

11

练习 1.7：编译一个包含不正确的嵌套注释的程序，观察编译器返回的错误信息。

练习 1.8：指出下列哪些输出语句是合法的（如果有的话）：

```
std::cout << "/*";
std::cout << "*/";
std::cout << /* "*/" */;
std::cout << /* "*/" /* "/*" */;
```

预测编译这些语句会产生什么样的结果，实际编译这些语句来验证你的答案（编写一个小程序，每次将上述一条语句作为其主体），改正每个编译错误。

1.4 控制流

语句一般是顺序执行的：语句块的第一条语句首先执行，然后是第二条语句，依此类推。当然，少数程序，包括我们解决书店问题的程序，都可以写成只有顺序执行的形式。但程序设计语言提供了多种不同的控制流语句，允许我们写出更为复杂的执行路径。

1.4.1 while 语句

while 语句反复执行一段代码，直至给定条件为假为止。我们可以用 **while** 语句编写一段程序，求 1 到 10 这 10 个数之和：

```
#include <iostream>
int main()
{
    int sum = 0, val = 1;
    // 只要 val 的值小于等于 10, while 循环就会持续执行
    while (val <= 10) {
        sum += val; // 将 sum + val 赋予 sum
        ++val;      // 将 val 加 1
    }
    std::cout << "Sum of 1 to 10 inclusive is "
               << sum << std::endl;
    return 0;
}
```

我们编译并执行这个程序，它会打印出

Sum of 1 to 10 inclusive is 55

与之前的例子一样，我们首先包含头文件 `iostream`，然后定义 `main`。在 `main` 中我们定义两个 `int` 变量：`sum` 用来保存和；`val` 用来表示从 1 到 10 的每个数。我们将 `sum` 的初值设置为 0，`val` 从 1 开始。

12

这个程序的新内容是 **while** 语句。**while** 语句的形式为

```
while (condition)
    statement
```

while 语句的执行过程是交替地检测 *condition* 条件和执行关联的语句 *statement*，直至 *condition* 为假时停止。所谓条件（*condition*）就是一个产生真或假的结果的表达式。只要 *condition* 为真，*statement* 就会被执行。当执行完 *statement*，会再次检测 *condition*。如果 *condition* 仍为真，*statement* 再次被执行。**while** 语句持续地交替检测 *condition* 和执行 *statement*，直至 *condition* 为假为止。

在本程序中，**while** 语句是这样的

```
// 只要 val 的值小于等于 10, while 循环就会持续执行
while (val <= 10) {
    sum += val; // 将 sum + val 赋予 sum
    ++val;      // 将 val 加 1
}
```

条件中使用了小于等于运算符（`<=`）来比较 `val` 的当前值和 10。只要 `val` 小于等于 10，条件即为真。如果条件为真，就执行 **while** 循环体。在本例中，循环体是由两条语句组